

Week 1 – Learning Report

Introduction

This was my first week of the internship. The focus was on the basics of how the web works and on the two core technologies used to build web pages, HTML and CSS. I already knew a little about these, but this week I went through them properly and tried small examples on my own. This report covers what I learned in the week.

1. Internet Basics – DNS, IP Address, HTTP

What I learned

I learned what actually happens behind the scenes when we type a website name in the browser. Earlier I only thought the browser "opens" the site, but there are a few steps involved before the page shows up.

IP Address

An IP address is a unique number given to every device connected to a network. It works like a home address, because data needs to know where to go and where it came from. An example is 192.168.1.1. This is the older IPv4 format. Since IPv4 addresses are running out, there is also IPv6, which is longer and looks like 2001:0db8:85a3::8a2e:0370:7334.

DNS

DNS stands for Domain Name System. Humans remember names like google.com easily, but computers only understand IP addresses. DNS converts the domain name into the IP address. I think of it like the contacts list on a phone: we tap a name, and the phone dials the number saved behind it.

What happens when we enter a URL

- I type a URL such as www.example.com in the browser and press Enter.
- The browser first checks if it already knows the IP for that name (cache). If not, it asks a DNS server.
- The DNS server replies with the IP address of the website's server.
- The browser sends an HTTP request to that server.
- The server sends back a response with the files, mainly HTML, CSS and JavaScript.
- The browser reads these files and displays the page.

HTTP

HTTP (HyperText Transfer Protocol) is the set of rules that the browser and server follow to talk to each other. The browser sends a request and the server sends a response. Some basic things I noted:

- Common methods: GET (to fetch data) and POST (to send data, like a login form).
- Status codes tell us what happened: 200 means OK, 404 means page not found, 500 means a server error.
- HTTPS is the secure version, where the data is encrypted. That is why we see the lock icon in the browser.

Why it is useful in web development

Knowing this helps me understand how websites load and why errors like 404 appear. It will also help later when I work with APIs, because API calls use the same request and response idea. It also helps in debugging, for example when a page does not load because of a wrong URL or a server problem.

Small example

I tried the ping command in the terminal to see the IP behind a domain name:

```
ping google.com
```

It showed an IP address along with the replies, which showed me that DNS had converted the name into a number. I also opened the Network tab in browser developer tools (F12) and saw the requests and status codes for a page.

2. HTML – Structure, Forms, Semantics

What I learned

HTML (HyperText Markup Language) is used to create the structure of a webpage. It does not make the page look good, it only tells the browser what content is on the page, such as headings, paragraphs, images and links. Styling is done later with CSS.

Basic HTML structure

Every HTML page follows a basic skeleton:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>My First Page</title>
  </head>
  <body>
    <h1>Hello World</h1>
  </body>
</html>
```

- <!DOCTYPE html> tells the browser that this is an HTML5 document.
- <head> holds information about the page, like the title and meta tags. It is not shown on the page.
- <body> holds everything that is visible to the user.

Common HTML elements

- Headings: <h1> to <h6>
- Paragraph: <p>
- Links:
- Images:
- Lists: , and
- Generic containers: <div> and

Forms and their purpose

Forms are used to take input from the user, for example login, signup, search or contact pages. The form sends the data to a server when the user submits it.

```
<form action="/submit" method="post">
```

```

<label for="name">Name:</label>
<input type="text" id="name" name="name" required>
<label for="email">Email:</label>
<input type="email" id="email" name="email">
<button type="submit">Submit</button>
</form>

```

- <input> has different types such as text, email, password, checkbox and radio.
- <label> describes the input field and makes the form easier to use.
- The "required" attribute stops the form from submitting if the field is empty.

Semantic HTML

Semantic HTML means using tags that describe the meaning of the content instead of using <div> everywhere. Some tags I learned:

- <header> – top part of the page, usually the logo or title
- <nav> – navigation links
- <main> – the main content of the page
- <section> – a group of related content
- <footer> – bottom part, like contact info or copyright

These are useful because the code becomes easier to read and understand. They also help search engines understand the page, and they help screen readers, so the page is more accessible for people with visual impairments.

Why it is useful in web development

HTML is the base of every website. Even if a project uses React or any other framework, it finally gets converted to HTML in the browser. Good structure and proper semantic tags make the work easier when the CSS and JavaScript are added later.

3. CSS – Selectors, Box Model, Flexbox

What I learned

CSS (Cascading Style Sheets) is used to style HTML elements, like colors, fonts, spacing and layout. If HTML is the skeleton of the page, CSS is what makes it look good.

CSS selectors

A selector tells CSS which element to style.

- Element selector: p { color: black; } styles all paragraphs.
- Class selector: .card { padding: 10px; } styles elements with class="card". It can be used many times.
- ID selector: #title { font-size: 24px; } styles one element with id="title".
- Group selector: h1, h2 { font-family: Arial; } styles several elements together.
- Descendant selector: nav a { text-decoration: none; } styles links only inside nav.

I also learned that when two rules conflict, the more specific selector wins (ID is stronger than class, and class is stronger than element).

Box model

In CSS every element is treated as a box. The box has four parts, from inside to outside:

- Content – the actual text or image

- Padding – space between the content and the border
- Border – the line around the padding
- Margin – space outside the border, between this box and other elements

```
.box {
  width: 200px;
  padding: 15px;
  border: 2px solid black;
  margin: 20px;
  box-sizing: border-box;
}
```

One useful thing I learned is box-sizing: border-box. Without it, padding and border get added on top of the width, so the box becomes bigger than expected. With border-box, the width includes the padding and border, which makes layouts easier to control.

Flexbox

Flexbox is a layout method used to arrange items in a row or column and to align them easily. It is applied on a parent container, and the children become flex items.

```
.container {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 10px;
}
```

- display: flex – turns on flexbox.
- flex-direction – row (default) or column.
- justify-content – aligns items along the main axis (start, center, space-between).
- align-items – aligns items along the other axis (for example, vertical centering).
- flex-wrap – lets items go to the next line when there is no space.

I used flexbox to make a simple navigation bar with the logo on the left and links on the right. It was much easier than older methods like floats.

How these help in designing webpages

Selectors help me target exactly the elements I want to style. The box model helps me manage spacing so that elements do not look cramped or overlapping. Flexbox helps me build layouts like navbars, card rows and centered content, and it also makes the page adjust better on different screen sizes.